2nd term

2019- 2018

الفرقة الثانية كهرباء

الفارس

Java

NEW 2019

نعمل لخدماتكم منذ عام 2000

العنوان : كوبري الجامعة - موقف منيا القمح - أسفل قاعة علاء الدين

للإستعلام ت / 01221732228 — 01016772044

السنتر ت :- 01098333260

للشكاوى والمقترحات :- 01224232288

للتواصل عبر الفيس / Al Fares

INHERITANCE

Object-oriented programming allows you to define new classes from existing classes. This is called inheritance.

البرمجة الموجهه تسمح لك بتعريف فصيله جديده من الفصائل التى تم تعريفها سابقاً وهذا يسمى الوراثه

الوراثه تعنى أن كل المتغيرات والدوال الموجوده داخل الفصيله الموروثه تم نقلها إلى الفصيله الوارثه

متى نستخدم الوراثه ؟؟؟

نستخدم الوراثه عندما يكون هناك class تحتوى على متغيرات (attributes) و دوال (methods) ونريد أستخدامها مره أخرى داخل فصيله تانيه فبدلاً من كتابتها مره أخرى نلجا إلى الوراثه

ملاحظه الفصيله الموروثه تسمى super class أو parent class أو base class

والفصيله الوراثه تسمى sub class أو child class او derived class

كيف يتم عمل الوراثه داخل البرمجه؟؟؟

يتم تعريف الclass الاولى كما تم التعريف سابقا وعمل الدوال والمتغيرات لها عادى

ثانيا عند تعريف الفصيله الثانيه يتم أضافه extends ثم أسم الفصيله الموراثه

مثال أذا كان عندنا فصيله تسمى Circle وفصيله تسمى Cylinder ونريد أن Cylinder ترث ال Circle فيكون بالشكل التالى

مثال 1

تكون فصيله ال Circle

```
public class Circle {

}
```

وتكون فصيله ال Cylinder

```
public class Cylinder extends Circle {

}
```

وبكدا تكون Cylinder ورثت ال Circle

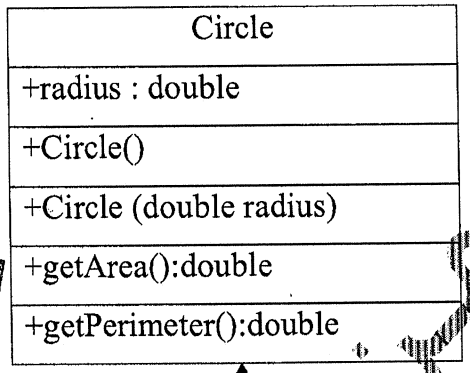
Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01224232288

(1)

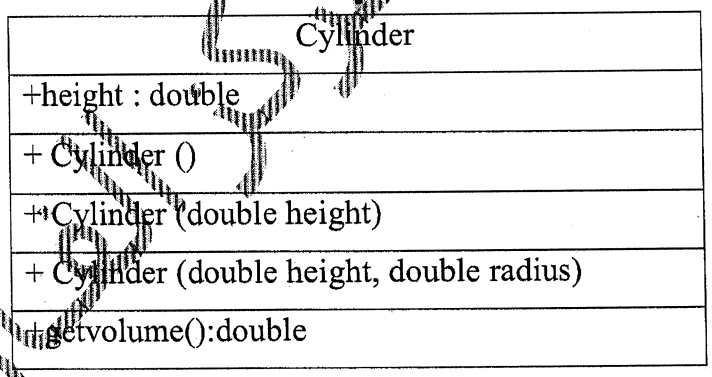
بمعنى أخر Cylinder هى ال Sub class وال Circle هى ال Super Class

مثال 2 أذا كان عندك فصيله تسمى GeometricObject تحتوى على مجموعة من الخصائص مثل color وFilled و DateCreated وبعض الدوال الموجوده كما ب UML التالى وكل الاشكال الهندسية لها هذة المتغييرات والدوال بالاضافة إلى مجموعة اخرى من المتغيرات والدوال فنجعل ال GeometricObject class هى ال Base Class وأى شكل هندسى أخر هو عبارة عن sub class

ال UML للمثال الاول



Cylinder Inherit Circle
Cylinder extends Circle
ال cylinder ترث ال Circle

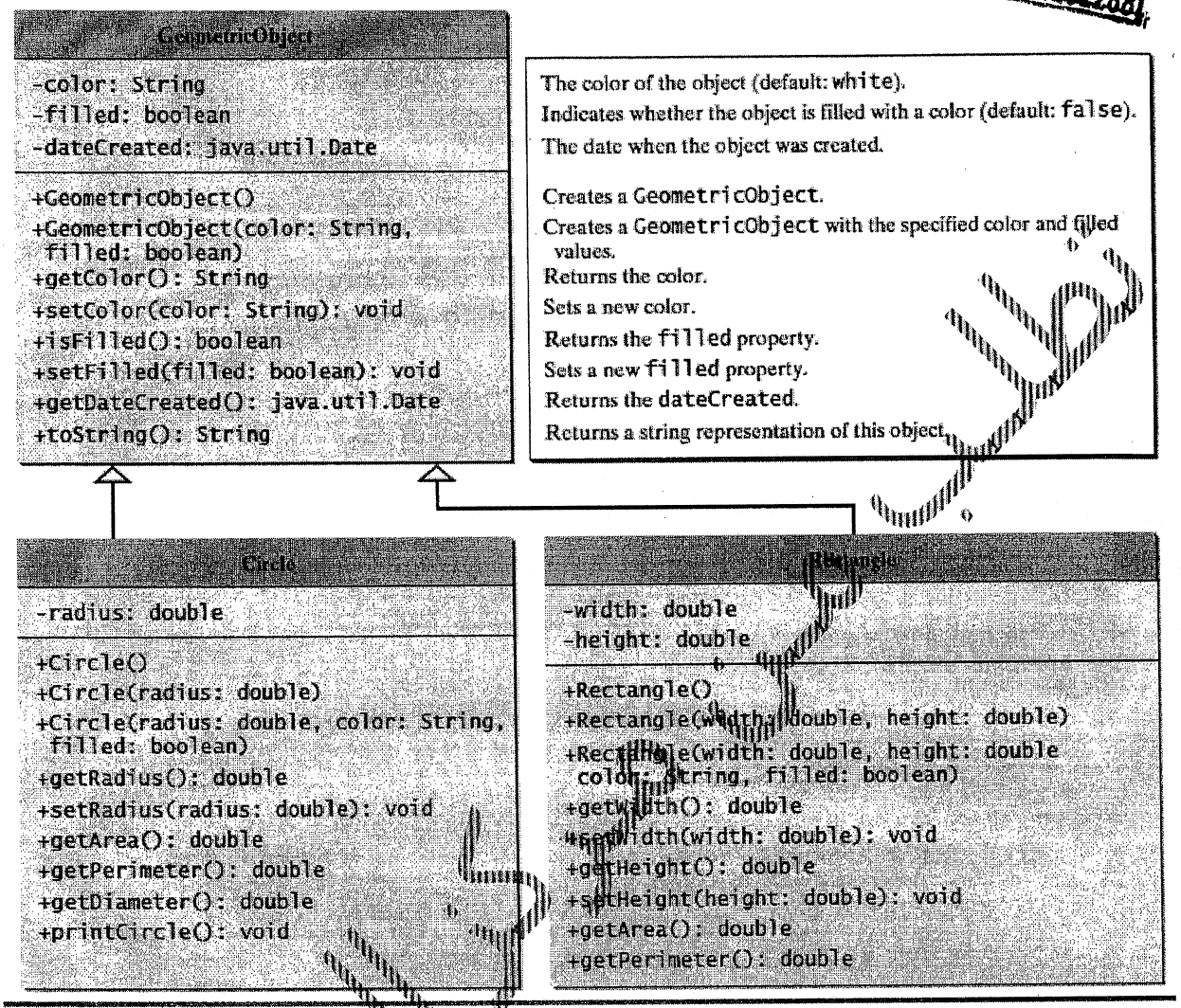


ال UML للمثال الثانى

Center Elfares
كوبرى الجامعة . أسفل قاعة علاء الدين
01221732220 - 01224232288

(2)

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01224232288



تعريف المثال الاول

فيتم تعريف الCircle Class بالطريقه التاليه

```
public class Circle {
    double raduis;
    Circle(){

    }
    Circle(double raduis){
```

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01224232288

```
    this.raduis=raduis;
  }
  double getArea(){
    return Math.PI*raduis*raduis;
  }
  double getPerimeter(){
    return 2*Math.PI*raduis;
  }
}
```

Center Elfares
كوبرى الجامعة . أسفل قاعة علاء الدين
01221732228 - 01224222288

ويتم تعريف ال Cylinder Class بالطريقه التاليه

```
public class Cylinder extends Circle {
  double height;
  Cylinder(){

  }
  Cylinder(double height){
    this.height=height;
  }
  Cylinder(double height,double raduis){
    this.height=height;
    this.raduis=raduis;
  }
  double getVolume(){
    return getArea()*height;
  }
}
```

Center Elfares
كوبرى الجامعة . أسفل قاعة علاء الدين
01221732228 : 01224222288

ويتم تعريف المثال الثانى بالشكل التالى

فتكون ال GeometricObject Class بالشكل التالى

```
import java.util.Date;
public class GeometricObject {
   private String color;
   private boolean filled;
   private Date dateCreated;

   public GeometricObject() {
      dateCreated = new Date();
   }

   public GeometricObject(String color, boolean filled) {
      dateCreated = new Date();
      this.color = color;
      this.filled = filled;
   }

   public String getColor() {
      return color;
   }

   public boolean isFilled() {
      return filled;
   }

   public void setColor(String color) {
      this.color = color;
   }
```

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732220 - 01224233288

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732220 - 01224233288

```
  public void setFilled(boolean filled) {
    this.filled = filled;
  }

  public Date getDateCreated() {
    return dateCreated;
  }
  public String toString() {
    return "created on " + dateCreated + "\ncolor: " + color +" and filled: " + filled;
  }

}
```

Center Elfares
كوبري الجامعة - أسفل قاعة علاء الدين
01221732220 - 01224232288

وتكون ال Circle Class بالشكل التالي

Subclass Superclass

```
public class Circle extends GeometricObject
```

```
public class Circle extends GeometricObject {
  private double raduis;

  public Circle(){
  }

  public Circle(double raduis) {
    this.raduis = raduis;
  }

  public Circle(double raduis, String color, boolean filled) {
```

Center Elfares
كوبري الجامعة - أسفل قاعة علاء الدين
01221732220 - 01224232288

```
    setColor(color);
    setFilled(filled);
    this.raduis = raduis;
  }

  public double getRaduis() {
    return raduis;
  }

  public void setRaduis(double raduis) {
    this.raduis = raduis;
  }
  public double getArea() {
    return raduis * raduis * Math.PI;
  }

  public double getDiameter() {
    return 2 * raduis;
  }

  public double getPerimeter() {
    return 2 * raduis * Math.PI;
  }

  public void printCircle() {
    System.out.println("The circle is created " + getDateCreated() + " and the radius is "
+ raduis);
```

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221792220 : 01224999988

تطلب من مركز الفارس

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221792220 : 01224999988

```
    }
}
```

وتكون ال Rectangle Class بالشكل التالى

Subclass Superclass

```
public class Rectangle extends GeometricObject
```

```
public class Rectangle extends GeometricObject {
    private double width;
    private double height;

    public Rectangle() {
    }

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    public Rectangle(double width, double height, String color, boolean filled) {
        setColor(color);
        setFilled(filled);
        this.width = width;
        this.height = height;
    }

    public double getWidth() {
```

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01224232288

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01224232288

```
    return width;
  }

  public void setWidth(double width) {
    this.width = width;
  }

  public double getHeight() {
    return height;
  }

  public void setHeight(double height) {
    this.height = height;
  }
  public double getArea() {
    return width * height;
  }

  public double getPerimeter() {
    return 2 * (width + height);
  }
}
```

Center Elfares
كوبرى الجامعة . أسفل قاعة علاء الدين
01221732228 - 01224232288

Center Elfares
كوبرى الجامعة . أسفل قاعة علاء الدين
01221732228 - 01224232288

الملاحظات على المثال الاول

فنلاحظ أننا فى ال Cylinder Class تم أستخدام الدوال والمتغيرات الموجوده فى Circle Class فـ Cylinder Class تحتوى على متغير height بالاضافه إلى ال radius الذى تم وراثته من ال Circle Class هو غير مكتوب ولكن موروث من فصيله أخرى فهو عند الفصيله الوارثه.

الملاحظات على المثال الثانى

نلاحظ أننا فى المثال الثانى فى ال GeometricObject Class تم تعريف متغييرات ودوال ثم أستخدامها داخل Circle Class وإيضاً داخل Rectangle Class كلها يمكن الوصول اليها ماعدا أذا كان المتغيير private فيتم الوصول اليه عن طريق دوال ال setter and getter له

Center Elfares
كوبرى الجامعة . أسفل قاعة علاء الدين
01221739999 : 01991999999

كيفية يتم عمل object من الفصيله الموروثه والوارثه

المثال الاول

```
Cylinder C3 = new Cylinder();
```

فى هذة الحاله يقوم باستدعاء داله البناء (constructor) الموجود داخل الفصيله الموروثه والتى لا تاخذ أى parameters

```
Cylinder C4 = new Cylinder(4);
```

فى هذة الحاله يقوم باستدعاء داله البناء (constructor) الموجود داخل الفصيله الموروثه والتى تاخذ parameters واحد وهو ال height

```
Cylinder C5 = new Cylinder(4,7);
```

فى هذة الحاله يقوم باستدعاء داله البناء (constructor) الموجود داخل الفصيله الموروثه والتى تاخذ اتنين parameters واحد المعرف داخل الفصيله وهو ال height والاخر الموروث من الفصيله Circle وهو ال radius

نلاحظ فيما سبق أننا لم نقم بتعريف المتغيرات ع انها private فماذا يحدث أذا كانت المتغيرات الموجوده داخل الفصيله الموروثه (Super Class) من النوع private ومن المعروف ان هذا النوع من البيانات لا يمكن الوصول اليه من خارج ال Class الذى تم تعريفه داخلها ولذلك عند عمل المتغيرات private فيوجد أحدى حلين

الحل الاول :- وهو عمل دوال Setter and Getter للمتغيرات ال private والتعامل من خلالها فى ال Sub Class

فيكون شكل المتغير فى Super Class بالشكل التالى

```
private double raduis;
  public void setRaduis(double raduis) {
    this.raduis = raduis;
  }
```

Center Elfares
كوبرى الجامعة . أسفل قاعة علاء الدين
01991799999 01224232288

ويكون شكل الاستخدام فى Sub Class بالشكل التالى

```
Cylinder(double height,double raduis){
```

```
    this.height=height;
    setRaduis(raduis);
  }
```

فتم الاستخدام عن طريق داله setter للمتغير

الحل الثانى :- يتم تعريف المتغيرات ع انها Protected بدلاً من Private وهو يمكن الوصول اليه من خلال أى Class داخل package وكذلك من خلال ال Sub Class مباشرة فقط (أى لا يمكن الوصول إليه من أى فصيله خارج ال Package)

فيكون التعريف داخل ال Super Class بالشكل التالى

```
protected  double raduis;
```

ويكون شكل الاستخدام فى ال Sub Class بالشكل التالى

```
Cylinder(double height,double raduis){
    this.height=height;
    this.raduis=raduis;
  }
```

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732220 - 01224232288

فتم إستخدام المتغير مباشرة دون الحاجه إلى داله Setter

فى المثال التانى

```
Rectangle r1 = new Rectangle();
```

فى هذا المثال قام بانشاء object من Rectangle class وقام باستدعاء ال no-args Construstor

```
Rectangle r2 = new Rectangle(5,7);
```

فى هذا المثال قام بانشاء object من Rectangle class وقام باستدعاء ال Construstor الذى ياخد قيميتى ال width,height فقط

```
Rectangle r3 = new Rectangle(5,7,"Blue",true);
```

فى هذا المثال قام بانشاء object من Rectangle class وقام باستدعاء ال Construstor الذى ياخد قيم ال width,height الموجود فى الفصيلة وكذلك قام باستخدام ال Color وfilled الموجوده فى ال super class اللى هى GeometricObject وتم الوصول اليهم باستخدام دوال ال Setter لهم لانهم متغييرات private

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732220 - 01224232288

Using the super Keyword

The keyword super refers to the superclass and can be used to invoke the superclass's methods and constructors.

كلمه super تدل ع Super Class وتستخدم فى Sub Class وذلك لآستدعاء الدوال العاديه (Methods) ودوال البناء (Constructor)

ففى المثال الاول

```
Cylinder(double height,double raduis){
    Super(radius);
    this.height=height;
}
```

فى هذا المثال قام باستدعاء ال constructor الموجود داخل ال Super class اللى هى ال circle وال }وهو ال Constructor اللى بياخد ال radius ك argument

```
double getVolume(){
    return super.getArea()*height;
}
```

وهنا تم الوصول الى الدوال الموجوده داخل ال super class إيضاً باستخدام super وذلك يتم أذا كان هناك إيضاً داله موجوده داخل ال subclass بنفس الاسم يعنى موجود داخل ال Cylinder class داله تسمى getArea فلما نحتاج أن نستخدم الدالة الموجوده فى ال subclass نقوم باستدعائها مباشرة ولما نحتاج ان نستدعى الدالة الموجوده فى superclass نقوم باستدعائها باستخدام ال super operator

ففى المثال السابق تم الوصول إلى عناصر ال Super Class عن طريق Super

Constructor Chain

Constructor is used to construct an instance of a class. Unlike properties and methods, the constructors of a superclass are not inherited by a subclass. They can only be invoked from the constructors of the subclasses using the keyword super

دوال البناء تستخدم لعمل Object من ال Class كما تم شرح سابقاً وع عكس كل الدوال والمتغيرات فان داله البناء للفصيله Super Class لا يتم وراثتها فى Sub Class مثل باقى الدوال وانما فقط يتم أستدعائها باستخدام super word

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01224232288

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01224232288

A constructor may invoke an overloaded constructor or its superclass constructor. If neither is invoked explicitly, the compiler automatically puts super() as the first statement in the constructor

داله البناء تقوم باستدعاء داله البناء لل Super Class وأن لم يقم المبرمج باستدعائها يقوم ال Compiler باستدعائها عن طريق وضع كلمه super() فى أول سطر داخل داله البناء لل Sub Class

```
public ClassName() {
  // some statements
}
```

Equivalent

```
public ClassName() {
  super();
  // some statements
}
```

```
public ClassName(double d) {
  // some statements
}
```

Equivalent

```
public ClassName(double d) {
  super();
  // some statements
}
```

In any case, constructing an instance of a class invokes the constructors of all the super classes along the inheritance chain. When constructing an object of a subclass, the subclass constructor first invokes its superclass constructor before performing its own tasks. If the superclass is derived from another class, the superclass constructor invokes its parent-class constructor before performing its own tasks. This process continues until the last constructor along the inheritance hierarchy is called. This is called constructor chaining

داله البناء لل Sub Class تستدعى كل دوال البناء للفصائل Super Classes

داله البناء لل Sub Class تستدعى داله البناء للفصيله Super Class قبل ما تقوم بتنفيذ الاوامر الموجوده داخلها وكذلك لو الداله Super Class كانت هى الاخرى Sub Class للاخرى فتقوم باستدعاء داله البناء للاخرى قبل ما تقوم بتنفيذ الاوامر الموجوده بداخلها وهكذا حتى نصل الى ال Class الاساسيه

Example

```
public class Faculty extends Employee {
    public static void main(String[] args) {
        Faculty F1= new Faculty();
    }
}
```

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01224232288

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732220 - 01224232288

```
        public Faculty() {
                System.out.println("(4) Performs Faculty's tasks");
        }
}
```

عند عمل Object من هذة الفصيله Faculty يقوم باستدعاء ال Constructor ولكن لان هذة ال Class ترث فصيله أخرى Faculty extends Employee فقبل تنفيذ ما بداخل ال Constructor لل Facluty يقوم باستدعاء ال Constructor للفصيله ال Super Class وهى Employee

```
class Employee extends Person {
        public Employee() {
                this("(2) Invoke Employee's overloaded constructor");
                System.out.println("(3) Performs Employee's tasks ");
}

        public Employee(String s) {
                System.out.println(s);
        }
}
```

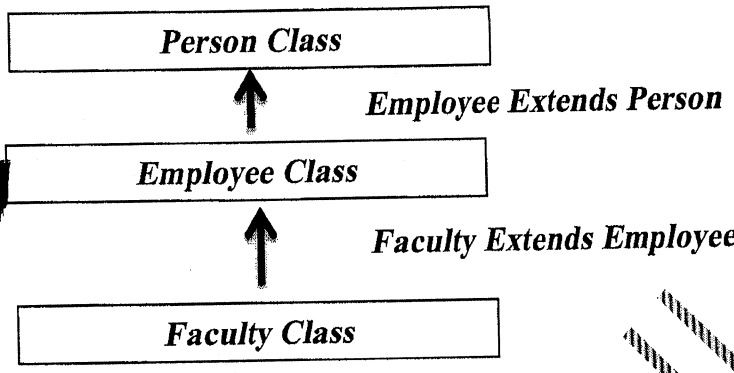
وعند أستدعاء ال Constructor للفصيله Employee نجد انها Sub Class أيضا لفصيله أخرى (Person) فقبل تنفيذ ال Constructor لل Employee يتم إستدعاء ال Constructor لل Person أولا

```
class Person {
        public Person() {
                System.out.println("(1) Performs Person's tasks");
        }
}
```

وعند إستدعاء constructor للPerson نجد انها لا ترث شي فيتم تنفيذه ال constructor لها اولا

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732220 - 01224232288

(14)



Center Elfares
كوبرى الجامعة . أسفل قاعة علاء الدين
01221732220 - 01224232208

الخلاصه عند عمل Object من فصيله Sub Class فان ال Constructor للفصيله Super بتم تنفيذه اولا

This:-

لاحظنا فى المثال السابق فى ال Employee Class انه أستخدم كلمه this وهى تستخدم لاستدعاء داله بناء من نفس الفصيله ولذلك يكون الخرج كالتالى

يتم تنفيذ ال Constructor لل Person اولا فيكون الخرج كالتالى

Performs Person's tasks

وبعد ذلك يتم تنفيذ ال Constructor للEmployee

ولكن non- parameter constructor يقوم باستدعاء parameter Constructor فيتم تنفيذه اولا ثم non- parameter فيكون الناتج التاى هكذا

Invoke Employee's overloaded constructor

Performs Employee's tasks

ثم يقوم بتنفيذ ال constructor للفصيله التى تم أنشاء ال object منها فيكون الخرج كالتالى

Performs Faculty's tasks

فى الحاله السابقه لم نقم نحن باستدعاء لل Constructor لل Super Class من ال Constructor لل Sub Class وذلك لانه كان يوجد Default Constructor داخل كل Class وعندما لا يقوم المبرمج باستدعاء ال Constructor من ال Sub Class لل Super Class بقوم JVM باستدعائه تلقائيا بوضع super(); ف كل Constructor لل Sub Class

```
class Person {
        public Person() {
                System.out.println("(1) Performs Person's tasks");
        }
```


Center Elfares
كوبرى الجامعة . أسفل قاعة علاء الدين
01221732220 - 01224232208

```
26    }
```

عندما يتم وراثه هذه الفصيله بالشكل التالى

```
11    class Employee extends Person {
12          public Employee() {
13                this("(2) Invoke Employee's overloaded constructor");
14                 System.out.println("(3) Performs Employee's tasks ");
15    }
16
17          public Employee(String s) {
18                System.out.println(s);
19          }
20    }
21
```

Center Elfares
كوبرى الجامعة . اسفل قاعة سيد الدين
01221732226 - 01224232288

فعند ترجمته الفصيله ال Employee باستخدام المبرمج يتم أضافه

```
11    class Employee extends Person {
12          public Employee() {
                  super();
13                this("(2) Invoke Employee's overloaded constructor");
14                 System.out.println("(3) Performs Employee's tasks ");
15    }
16
17          public Employee(String s) {
18                System.out.println(s);
19          }
20    }
21
```

Center Elfares
كوبرى الجامعة . اسفل قاعة سيد الدين
01221732226 - 01224232288

وهى التى تقوم باستدعاء ال Constructor للفصيله ال Super

ماذا لو كانت ال Super Class لاتحتوى ع Default Constructor فيجب ع المبرمج أضافة هو ال Super(parameters) داخل ال Constructor لل Sub Class

فلو كانت الفصيله Super بهذا الطريقه

Center Elfares
كوبرى الجامعة . أسفل قاعة علاء الدين
01221732220 : 01224232288

```
public class Circle  {
    double raduis;
   Circle(double raduis){
      this.raduis=raduis;
   }
```

فنلاحظ انها لا تحتوى الا ع Parameter Constructor فعند كتابه ال Sub Class لابد ع المبرمج أضافه ال super(parameter) والا ال يكون أول سطر فى ال Constructor فيكوم شكل ال Sub Class كالتالى

```
public class Cylinder extends Circle {
    double height;
    Cylinder(){
       super(5);
    }
    Cylinder(double height){
       super(6);
       this.height=height;
    }
    Cylinder(double height,double raduis){
       super(raduis);
       this.height=height;
    }
```

Center Elfares
كوبرى الجامعة . أسفل قاعة علاء الدين
01221732220 : 01224232288

The keyword super can also be used to reference a method other than the constructor in the super class. The syntax is:

super.method(parameters);

ويمكن إستخدام كلمه super وذلك لإستدعاء دوال موجوده فى Super Class مباشرة بوضع كلمه Super وأسم ال Method ثم ال Parameters لها

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01224232288

Overriding Methods

معناه اعاده كتابه Method مكتوبه داخل ال Super Class وتودى وظيفتها باكمل وجه ويتم أعاده كتابتها داخل ال Sub Class بنفس الاسم وبنفس ال Parameters وبنفس نوع القيمه الراجعه (Return Value)

To override a method, the method must be defined in the subclass using the same signature and the same return type as in its superclass

ويمكن إستدعاء ال Method الموجود داخل ال Super Class عن طريق وضع super.MethodName

An instance method can be overridden only if it is accessible. Thus a private method cannot be overridden, because it is not accessible outside its own class. If a method defined in a subclass is private in its superclass, the two methods are completely unrelated.

الدوال المعرفه كانها Private داخل ال Super Class لايمكن عمل لها Overriding داخل ال Subclass وذلك لانه لايمكن إستخدامها خارج ال Class المعرفه داخلها فلو تم عمل Method فى ال Subclass هى معرفه داخل Super Class ع انها Private فهما ليس لهم علاقه ببعضهما

Like an instance method, a static method can be inherited. However, a static method cannot be overridden. If a static method defined in the superclass is redefined in a subclass, the method defined in the superclass is hidden. The hidden static methods can be invoked using the syntax SuperClassName.staticMethodName

الدوال ال static المعرفه داخل ال Super Class لا يمكن عمل لها Over Ride داخل ال Sub Class واذا تم اعادة تعريفها داخل ال Sub Class فان الداله المعرفه داخل Super Class تكون مخفيه ولاستدعائها نقوم بكتابه الامر التالى

SuperClassName.staticMethodName

Example

```
double getArea(){
    return 2*super.getArea()+getPerimeter()*height;
}
```

المثال السابق يوضح إعادة تعريف داله getArea() فى ال Cylinder Class وهى معرفه داخل الCircle Class بنفس الاسم وعندما أردنا أستخدام الداله الموجوده داخل Circle تم وضع super.ClassName

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01224232288

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01221732288

Overriding vs. Overloading

Overloading means to define multiple methods with the same name but different signatures. Overriding means to provide a new implementation for a method in the subclass.

معني ال OverLoading انه يتم تعريف أكثر من داله بنفس الاسم ولكن نوع او عدد ال Parameter يكون مختلف كل واحده عن التانيه داخل نفس الفصيلة

اما ال Over Riding يتم تعريف اكثر من داله بنفس الاسم وبنفس ال Parameters ولكن الوظيفه تكون مختلفه داخل الفصائل الوراثة

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overrides the method in B
  public void p(double i) {
    System.out.println(i);
  }
}
```

(a)

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overloads the method in B
  public void p(int i) {
    System.out.println(i);
```

(b)

When you run the Test class in (a), both a.p(10) and a.p(10.0) invoke the p(double i) method defined in class A to display 10.0. When you run the Test class in (b), a.p(10) invokes the p(int i) method defined in class A to display 10, and a.p(10.0) invokes the p(double i) method defined in class B to display 20.0.

Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01221732288

Note the following:

- Overridden methods are in different classes related by inheritance; overloaded methods can be either in the same class or different classes related by inheritance.
- Overridden methods have the same signature and return type; overloaded methods have the same name but a different parameter list.